

Network & URL Layers Public DNS-based URLs

- ☐ (<https://test.com/api/>) Private IPs (192.xx.x.x) on internal networks
 - ☐ Why frontend code needs public URLs (browser security, CORS, HTTPS)
 - ☐ Why service-to-service can use internal IPs (faster, no TLS needed internally) Service names in Docker Compose / Kubernetes vs hardcoded IPs Reading 404 errors from IIS vs Spring Boot (HTML vs JSON tells you the source)
 - ☐ The local-vs-deployed config issue (defaults matter)
-

The Core Idea

Every service in a system has **multiple ways to be reached**, depending on where you're calling from. Same service, different addresses. Which one you use depends on who's calling.

Think of an office building:

- Outside visitors use the street address: "123 Main Street, Suite 401"
- Coworkers use the internal extension: "Ext. 4501"
- People at the same desk just say: "Hey, Sarah"

Same person, three ways to reach her. Use the right one for your context.

Services work the same way.

The Two Worlds

Outside world: Public URLs

<https://test.com/api/>

This is the "street address." Anyone on the internet can reach it. Goes through DNS, load balancers, encryption.

Inside world: Private IPs or service names

<http://192.165.xx.xxxx:8080/api/>

<http://cs-service:8080/api/> (Docker name)

<http://cs-service.namespace.svc.cluster.local/> (Kubernetes name)

This is the "internal extension." Only reachable from inside your network. No public exposure.

Same service. Two addresses. Pick the right one for who's calling.

The Rule: Match the Caller to the Address

Frontend (browser) → must use public URL

Why? The browser runs on a user's laptop, somewhere on the internet. From there:

- `192.165.x.x` is unreachable (it's *your* office's internal address, not theirs)
- HTTPS is required (browsers block mixed content)
- CORS expects a real domain, not a raw IP
- TLS certificates work for domains, not IPs

So frontend code calls `https://test.com/api/`. Always.

Backend → backend on the same network: use internal address

Why? When two services live in the same network:

- Internal address is faster (no DNS lookup, no internet hop)
- No TLS overhead needed (trusted network)
- Direct connection, fewer points of failure

So your Java service calls `http://cs-service:8080/api/`. Stays inside the network.

Real-world picture

User's laptop (browser)

`https://test.com/api/`

(goes through DNS, load balancer, ingress, into network)



Inside the company's private network

Java service

`http://cs-service:8080/api/`

(stays inside, fast, direct)



cs-service

Don't Hardcode IPs

Bad

`CS_SERVICE_URL: http://192.165.25.255:8080/api/`

Good

`CS_SERVICE_URL: http://cs-service:8080/api/`

Why?

IPs change. Containers restart with new IPs. Servers get replaced. Networks get reorganized. If your code says "talk to 192.165.xx.xxxx," it breaks the day someone changes anything.

Service names are stable. Docker Compose, Kubernetes — they all have internal DNS that resolves names to current IPs automatically. You call `cs-service`, the platform figures out where it lives.

Hardcoding IPs is a junior mistake. Using service names is the senior pattern.

Reading 404s

This was a real moment in our conversation. You got a 404 and the natural reaction is "endpoint doesn't exist."

But look at the *shape* of the response:

HTML page that says "Server Error" with Verdana font and IIS branding
→ IIS itself returned this. Request reached the server, but IIS couldn't route it.

JSON like {"timestamp": "...", "status": 404, "error": "Not Found"}
→ Spring Boot returned this. Request reached your application code.

Different sources of 404 mean different problems:

- **HTML 404 from IIS** = wrong URL path, request didn't even hit your application
- **JSON 404 from Spring** = your application ran but couldn't find the requested resource

Read the format, not just the status code. The shape of the response tells you which layer failed.

The Local-vs-Deployed Trap

This bit you hard. The pattern is:

```
# application.yml
cs-service:
  link: ${CS_SERVICE_URL:https://test-environment.com/cs/}
```

When you run locally, `CS_SERVICE_URL` env var isn't set. Spring uses the default: the test environment URL.

So your local dev quietly calls the test environment. No warning. No error. Just wrong behavior.

Senior rule: defaults should reflect the local-dev case. Make the default `http://localhost:5000/cs/`. Test/staging/prod always set the env var explicitly, so they never hit the default.

The default is for the case when nothing else is configured. Make it match that case.

Quick Reference

Calling from	Calling what	Use which address
Browser	Backend	Public HTTPS URL
Backend	Same-network backend	Service name (best) or internal IP
Backend	External service	Public HTTPS URL
Local dev	Local service	<code>localhost</code>
Same container	Same container	<code>localhost</code>

The One Thing to Remember

Match the address to the caller's network position. Frontend = public URL.
Backend-to-backend = internal address. Local dev = localhost.

When you see a connection error or wrong response, ask first: "where is the caller? where is the callee? am I using the right address for that path?"

Most network problems are actually just "wrong address for this context." Get that mental model, and you'll solve them in seconds instead of hours.

That's the senior view. Read it twice. Then go look at your project's URLs and ask: "is each caller using the right address for where it's calling from?" You'll find issues. You'll fix them. That's the work.